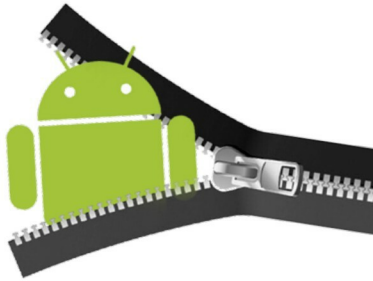


OPTIMISATION WITH ZIPALIGN



If you are an Android freak, you must be constantly searching for all kinds of mods for your device—maybe fun-themed apps, exciting themes or even custom ROMs! And what's most commonly associated with custom ROMs is Zipalign. The developers among you will be quite familiar with it, but probably not the newbies. So, in this article we're going to dwell on Zipalign!

To begin, *Zipalign* is an archive alignment tool for Android application files (*.apk* files). It was first introduced by Google with the Android 1.6 SDK. It optimises application packaging so that Android can interact with the app much more efficiently at runtime, thus making the overall system run faster.

How does it work? In Android, all data files are stored in the *.apk* file. These data files are accessed by multiple processes in the Android Operating Environment. First, the installer reads the manifest file to handle the associated permissions. Second, the *Home* application seeks the resources for the application name and icon. Apart from this, the system server needs access to these resources for background operations such as displaying notifications, updating and the like. And finally, the application itself makes use of the resource files. As Android is a GNU/Linux-based OS, memory mapping plays an important role in efficient process handling. It's pretty obvious that the Android resource-handling code would work more efficiently if resources are aligned on 4-byte boundaries, essentially by memory mapping them. In this way, Android doesn't need to go through the complete application package to get the desired manifest.

Why Zipalign apps?

For resources that are not Zipaligned, the resource-handling code has to fall back to slower access methods that consume a lot of additional memory. While this fall-back mechanism is good for application developers, allowing flexibility in development, the scene is different for users. The best case is during slow-downs, like when the *Home* application and the unaligned application both launch slower than they should. The worst case is when several unaligned applications are

installed—they occupy much more memory than they should, and thus the system crashes due to the constant starting and killing processes. The user is left with a slow device and frequent battery run-outs.

As mentioned earlier, *Zipalign* became part of the Android SDK from 1.6 onwards, so you don't need to install it separately. It's very easy to align resources in your application using one of three methods, which we will cover below. First, please note that you should do the Zipalign operation only after you have signed your application package with your own private key. Zipaligning it before signing means the alignment will be disturbed by the signing. The same goes for any other operation that may change package contents. So, it is preferable to Zipalign the package as the last step, and even better, to do it manually. Then perform the verifying operation, as given below.

Aligning using ADT

The ADT plug-in for Eclipse (v0.9.3 onwards) automatically aligns 'release' application packages when the *Export Wizard* is used to create them—right-click the project, and select *Android Tools > Export Signed Application Package*. Alternatively, access the export wizard via the first page of the *AndroidManifest.xml* editor.

Aligning manually

You can manually run zipalign (from the SDK *tools/* folder, if you haven't added the folder to your *\$PATH*) with the following command:

```
zipalign [-f] [-v] <alignment> inputfile.apk outputfile.apk
```